

Data Analysis and Programming

Joanna Wang

Peking University HSBC Business School

Big Data in Finance

Why Big Data Matters in Finance

Improved Decision-Making

- Predict market trends and customer behavior
- Real-time portfolio monitoring and stress testing
- Credit risk scoring with alternative data

Operational Efficiency

- Automate compliance checks and fraud detection
- Customer segmentation for personalized services
- American Express reduced fraud by 60% with predictive analytics

Risk Management

- Cross-check transactions against behavioral patterns
- Detect money laundering and insider trading signals
- ESG data integration for sustainable investing

Regulatory Compliance

- Detect and report suspicious activities in real time
- Automated regulatory reporting (MiFID II, Basel III)
- Audit trails from structured and unstructured sources

The Big Data Pipeline

Step 1 — Data Acquisition

Sources: SEC filings, social media, sensor networks, satellite imagery, transaction records

Characterized by **Volume** (scale), **Velocity** (speed), and **Variety** (formats)

Step 2 — Preprocessing

Cleaning, normalizing, transforming, and aggregating raw data into analyzable format

Step 3 — Analysis

Statistical models, machine learning, NLP, and LLMs to uncover patterns and insights

Step 4 — Application

Trading signals, risk scores, regulatory reports, research findings, or product recommendations

Industry Use Case — JP Morgan and Chase

- Analyzed income and spending habits of ~2.5 million account holders (2012–2014)
- Goal: uncover earning and spending patterns for consumer behavior research
- Today: real-time fraud detection, credit scoring, macro forecasting, and GPT-4-powered financial advisor chatbot

The analytics systems at JPMorgan use customers' transactions data and combine them with other data sources to draw better insights and get a clear perspective of which customers are credit-worthy and prospective buyers of financial products or services. The investment banking behemoth uses big data technology to crunch massive amounts of data to detect patterns in customer behavior, helping the bank identify market risks and possible opportunities to make money.

JPMorgan utilizes predictive analytics to help its customers manage their working capital and cash forecasting needs. The bank's forecasting services leverage predictive analytics to determine alterations in limits and credit lines to optimize cash balances.

Industry Use Case — Goldman Sachs

- ML models create dynamic pipelines that adapt to different data formats
- Covers more companies than traditional fundamental analysis
- Applications: equity research automation, risk factor modeling, alternative data signals

The bank strongly focuses on creating data-driven investment models that evaluate thousands of companies globally to find the right investing opportunities. Analyzing large datasets, including financial statements and market data like prices, returns, etc., helps the firm identify strong businesses with attractive valuations and make the right investing decisions.

In a [recent report](#), Takashi Suwabe, a senior portfolio manager at Goldman Sachs Asset Management, said that computers could only analyze organized or easily quantifiable data in the past. However, with the inception of big data technologies, they can also analyze unstructured data. These innovations enable them to interpret data from different formats, including images and speech.

Industry Use Case — American Express

- Cross-checks real-time transactions against stored behavioral profiles
- Result: reduced fraudulent transactions by **60%** (Harvard Business School study)
- Models flag suspicious patterns *before* the transaction completes
- Also uses big data for merchant recommendations and churn prediction

Analyzing the vast amounts collected from cardholders and merchants, the credit card giant makes fraud assessments in fractions of a second. They ensure a hassle-free purchasing experience for legitimate customers while limiting fraudulent transactions. This data includes purchase history, monthly spending, location data, and more. By processing this data, they comprehensively view the cardholder's spending behavior.

Data scientists at American Express have created a tool called Enhanced Authorization (EA), allowing American Express and merchants to identify who is behind the transaction. During any transaction, the merchant sends additional information such as email address, IP address, and shipping address.

Industry Use Case — Morgan Stanley

- Invested ≈\$2.7 billion in AI, ML, and big data (2021)
- Extracts insights and hidden patterns from alternative data sources
- Deployed AI chatbot (GPT-4 powered) to assist financial advisors in client interactions
- Helps advisors locate relevant research and personalize recommendations

Morgan Stanley also leverages big data and AI/ML for fraud detection. In a [conversation with Times of India](#) in 2022, Katherine Wetmur, Co-CTO at Morgan Stanley, said that they use AI/ML to predict fraudulent events and send alerts. The company added more information to ensure their operations staff could do more triaging and reduce the response time.

Research Data — Public Firm Disclosures

- SEC EDGAR: annual and quarterly filings (10-K, 10-Q, 8-K) for all US public firms
- Rich textual data: risk factors, MD&A, earnings guidance
- Researchers apply NLP tools to extract tone, uncertainty, specificity, and AI mentions

CIMPRESS PLC
ANNUAL REPORT ON FORM 10-K
For the Year Ended June 30, 2024

TABLE OF CONTENTS

Part I	Page
Item 1. Business	1
Item 1A. Risk Factors	13
Item 1B. Unresolved Staff Comments	24
Item 1C. Cybersecurity	25
Item 2. Properties	25
Item 3. Legal Proceedings	26
Item 4. Mine Safety Disclosures	26

UNITED STATES
SECURITIES AND EXCHANGE COMMISSION
Washington, D.C. 20549

Form 10-K

(Mark One)

☒ ANNUAL REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE ACT OF 1934
For the fiscal year ended JUNE 30, 2024

or

☐ TRANSITION REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE ACT OF 1934
For the transition period from _____ to _____

Commission file number 001-37459

Cimpress plc

Research Data — SEC EDGAR Full-Text Search

- Machine-readable full-text search system
- Example studies: Li (2010) on risk language; Hassan et al. (2019) on political risk; Wang et al. (2025) on AI disclosure
- APIs: sec-edgar-downloader, WRDS, Bloomberg

PART I

Item 1. Business

Overview & Strategy

Cintas is a strategically focused collection of businesses that specialize in print mass customization, through which we deliver large volumes of individually small-sized customized orders of printed materials and related products. Our products and services include a broad range of marketing materials, business cards, signage, promotional products, logo apparel, packaging, books and magazines, wall decor, photo merchandise, invitations and announcements, design and digital marketing services, and other categories. Mass customization is a core element of the business model of each Cintas business and is a competitive strategy that seeks to produce goods and services to meet individual customer needs with near mass production efficiency. We discuss mass customization in more detail further below.

We have grown substantially over our history, from \$0 in 1995 to \$0.2 billion of revenue in fiscal year 2006, the year when we became a publicly traded company, then to \$3.3 billion of revenue in fiscal year 2024. As we have grown we have achieved important benefits of scale. Our strategy is to invest in and build customer-focused, entrepreneurial print mass customization businesses for the long term, which we manage in a decentralized, autonomous manner. We draw competitive advantage across Cintas through a select, few shared strategic capabilities that have the greatest potential to create Cintas-wide value. We limit all other central activities to only those which absolutely must be performed centrally.

Risks Related to Our Corporate and Capital Structures

Our credit facility and the indentures that govern our notes restrict our current and future operations, particularly our ability to respond to changes or to take certain actions.

Our senior secured credit facility that governs our Term Loan B and revolving credit and the indentures that govern our 7.0% Senior Notes due 2026, which we collectively refer to as our debt documents, contain a number of restrictive covenants that impose significant operating and financial restrictions on us and may limit how we conduct our business, execute our strategy, compete effectively, or take advantage of new business opportunities, including restrictions on our ability to:

- incur additional indebtedness, guarantee indebtedness, and incur liens
- pay dividends or make other distributions or repurchase or redeem capital stock
- prepay, redeem, or repurchase subordinated debt
- issue certain preferred stock or similar redeemable equity securities
- make loans and investments

Research Data — Investors' Footprint on SEC

- SEC server logs record every IP address that downloads filings
- Researchers reconstruct institutional investor *attention* from download patterns
- Drake et al. (2015): EDGAR search activity predicts future abnormal returns
- Also available: 13F holdings, insider trading filings (Form 4)

EDGAR Log File Data Sets

EDGAR log file data sets provide information on internet search traffic for EDGAR filings through SEC.gov. The data sets contain information in CSV format extracted from log files from the EDGAR Archive on SEC.gov, and the information can be used to infer user access statistics.

There are two EDGAR log file data sets. The first set pertains to search traffic from January 1, 2003 through June 30, 2017. The second set pertains to search traffic from May 19, 2020 forward. EDGAR log file data for July 1, 2017 to May 18, 2020 is no longer available.

Research Data — USPTO Patent Data

- US Patent and Trademark Office publishes full application data in XML format
- Millions of records: filing date, applicant firm, examiner, claims text, abstract
- Used to measure firm-level innovation, inventor mobility, and technology diffusion
- Linked to CRSP/Compustat via PERMNO matching

ISSUED PATENTS (PATENT GRANTS)

[Top]

Patent Official Gazettes (JUL 2, 2002 - PRESENT)

Contains bibliographic (front page) information, a representative claim, and a drawing (if applicable) of each patent grant issued that week (Tuesdays). Includes U.S. Patent and Trademark Office Notices which provide important information and changes in rules concerning both patents and trademarks.

Patent Official Gazettes (JAN 2024 - DEC 2024)

Patent Official Gazettes (JAN 2023 - DEC 2023)

Patent Official Gazettes (JAN 2022 - DEC 2022)

Patent Official Gazettes (JAN 2021 - DEC 2021)

Patent Official Gazettes (JAN 2020 - DEC 2020)

Patent Official Gazettes (JAN 2019 - DEC 2019)

Patent Official Gazettes (JAN 2018 - DEC 2018)

Patent Official Gazettes (JAN 2017 - DEC 2017)

Patent Official Gazettes (JAN 2016 - DEC 2016)

Patent Official Gazettes (JAN 2015 - DEC 2015)

Patent Official Gazettes (JAN 2014 - DEC 2014)

Patent Official Gazettes (JAN 2013 - DEC 2013)

Patent Official Gazettes (JAN 2012 - DEC 2012)

Patent Official Gazettes (JAN 2011 - DEC 2011)

Patent Official Gazettes (JAN 2010 - DEC 2010)

Patent Official Gazettes (JAN 2009 - DEC 2009)

Patent Official Gazettes (JAN 2008 - DEC 2008)

Patent Official Gazettes (JAN 2007 - DEC 2007)

Patent Official Gazettes (JAN 2006 - DEC 2006)

Patent Official Gazettes (JAN 2005 - DEC 2005)

Patent Official Gazettes (JAN 2004 - DEC 2004)

Patent Official Gazettes (JAN 2003 - DEC 2003)

Patent Official Gazettes (JUL 2, 2002 - DEC 2002)

Research Data — TAQ and Conference Calls

Daily TAQ (Trade and Quote) provides users access to all trades and quotes for all issues traded on NYSE, Nasdaq and the regional exchanges for the previous trading day. It's a comprehensive history of daily activity from NYSE markets and the U.S. Consolidated Tape covering all U.S. Equities instruments (including all CTA and UTP participating markets).

The file contains the Master File with Primary Exchange Indicator, the "NBBO file" containing continuous National Best Bid and Offer updates, and consolidated trades and quotes for all listed and non-listed issues. Historical information is available for an additional fee. Daily TAQ data reflects Trades and Quotes reported to the SIP between 4:00am-8:00pm.

1.1 MARKETS COVERED

The Daily TAQ product covers:

- All CTA Participating Markets – Tapes A, B and C
- NYSE
- Nasdaq (OTC)
- All Regionals Exchanges

1.2 INSTRUMENTS COVERED

The Daily TAQ product covers Equities.

NYSE TAQ: millisecond-level trade and quote data

NYSE TAQ Database

- Millisecond-level trade and quote data for all US equities
- Used for liquidity, price impact, and market microstructure research

Earnings Conference Calls

- Quarterly interaction between management and analysts
- Rich Q&A session: information asymmetry, insider communication
- Sources: Bloomberg, Capital IQ, WRDS

Python for Data Analysis

Python Ecosystem for Finance Research

Data Acquisition

- `requests`, `urllib` — HTTP requests and downloads
- `BeautifulSoup`, `lxml` — web scraping and XML parsing
- `sec-edgar-downloader` — bulk SEC filings
- `selenium` — browser automation for dynamic pages

Data Wrangling

- `pandas` — DataFrames, merging, groupby, time series
- `numpy` — fast numerical arrays
- `polars` — memory-efficient alternative to `pandas`

Text and NLP

- `nlTK`, `spacy` — tokenization, POS tagging
- `sentence-transformers` — dense text embeddings
- `openai`, `anthropic` — LLM APIs (covered later)

Machine Learning

- `scikit-learn` — classical ML, preprocessing, CV
- `xgboost`, `lightgbm` — gradient boosting
- `statsmodels` — OLS, panel models, time series

Web Scraping — Downloading Bulk Data Files (urllib + BeautifulSoup)

```
import urllib.request
from bs4 import BeautifulSoup
import re, os
from zipfile import ZipFile

base_url = "https://bulkdata.uspto.gov/data/patent/application/redbook/fulltext/2023/"
save_dir = "data/patents/"
os.makedirs(save_dir, exist_ok=True)

# Parse the server directory listing to discover .zip file links
html = urllib.request.urlopen(base_url).read()
soup = BeautifulSoup(html, "lxml")

for link in soup.find_all("a", href=True):
    href = link["href"]
    if re.search(r"\.zip$", href):
        out_path = os.path.join(save_dir, href)
        print(f"Downloading {href} ...")
        urllib.request.urlretrieve(base_url + href, out_path)

# Unzip all downloaded archives
for fname in os.listdir(save_dir):
    if fname.endswith(".zip"):
        with ZipFile(os.path.join(save_dir, fname)) as z:
            z.extractall(save_dir)
            print(f"Extracted {fname}")
```


Parsing XML Data — USPTO Patent Structure

```
ipw230216.xml X
P: > Google Drive > shortcut-targets-by-id > 1j0f0Rqf75JhZC3bumNhlm0Kc/htmO > Labor_Saving_Innovation > USPTO raw data 9 2023 > app
1  <?xml version="1.0" encoding="UTF-8"?>
2  <DOCINFO us-patent-application SYSTEM "us-patent-application-v06-2022-02-17.dtd" [ ]>
3  <us-patent-application lang="en" dtd-version="v4.6 2022-02-17" file="US20230045788A1-20230216.XML">
4  <us-bibliographic-data-application lang="en" country="US">
5  <publication-reference>
6  <document-id>
7  <country>US</country>
8  <doc-number>20230045788</doc-number>
9  <kind>A1</kind>
10 <date>20230216</date>
11 </document-id>
12 </publication-reference>
13 <application-reference appl-type="utility">
14 <document-id>
15 <country>US</country>
16 <doc-number>17976884</doc-number>
17 <date>20221031</date>
18 </document-id>
19 </application-reference>
20 <us-application-series-code>17</us-application-series-code>
21 <classifications-ipc>
22 <classification-ipc>
23 <ipc-version-indicator><date>20060101</date></ipc-version-indicator>
24 <classification-level>A</classification-level>
25 <section>B</section>
26 <class>6</class>
27 <subclass>B</subclass>
28 <main-group>7</main-group>
29 <subgroup>1</subgroup>
30 <symbol-position>F</symbol-position>
31 <classification-value>1</classification-value>
32 <action-date><date>20230216</date></action-date>
33 <generating-office><country>US</country></generating-office>
34 <classification-status>B</classification-status>
35 <classification-data-source>N</classification-data-source>
36 </classification-ipc>
```

Parsing XML Data — Extracting Fields with BeautifulSoup

```
import pandas as pd
from bs4 import BeautifulSoup

records = []

with open("ipa230101.xml", "r", encoding="utf-8") as f:
    soup = BeautifulSoup(f, "lxml-xml")

for patent in soup.find_all("PatentData"):
    records.append({
        "filing_number": patent.findtext("ApplicationNumberText", default=""),
        "filing_date":   patent.findtext("FilingDate",          default=""),
        "examiner":      patent.findtext("PersonFullName",      default=""),
        "abstract":      patent.findtext("AbstractText",         default=""),
    })

df = pd.DataFrame(records)
df["filing_date"] = pd.to_datetime(df["filing_date"], errors="coerce")
print(f"Parsed {len(df):,} patent records")
df.to_csv("patents_parsed.csv", index=False)
```

Pandas — Core Data Wrangling Operations

```
import pandas as pd

# Load and inspect
df = pd.read_csv("patents_parsed.csv", parse_dates=["filing_date"])
print(df.shape, df.dtypes)

# Filter and derive new variables
df = df[df["filing_date"].dt.year >= 2010].copy()
df["year"] = df["filing_date"].dt.year

# Aggregate: patent count per examiner per year
examiner_counts = (df
    .groupby(["examiner", "year"])["filing_number"]
    .count()
    .reset_index(name="patent_count"))

# Merge with firm-level data and export to Stata
firm_data = pd.read_csv("firm_crosswalk.csv")
merged = examiner_counts.merge(firm_data, on="examiner", how="left")
merged.to_stata("analysis_data.dta", write_index=False)
```

Natural Language Processing (NLP) and Text Analysis

From Text to Numbers

- Computers cannot directly process raw text — we need **numeric representations**
- **Sparse vectors (classical):**
 - Bag-of-Words, TF-IDF
 - Each dimension = one vocabulary word
 - Ignores word order and semantics
- **Dense vectors (modern):**
 - Word2Vec, BERT, sentence embeddings
 - Captures semantic meaning
 - Similar meaning $\Rightarrow$ similar vectors

	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

Sentence Embeddings with sentence-transformers (runs locally)

```
from sentence_transformers import SentenceTransformer
import pandas as pd, numpy as np

# Load a pre-trained model (downloads automatically on first run)
model = SentenceTransformer("all-MiniLM-L6-v2") # fast, 384-dim vectors
# Finance-specific alternatives: "ProsusAI/finbert", "yiyanghkust/finbert-tone"

df = pd.read_csv("patents_parsed.csv")
abstracts = df["abstract"].fillna("").tolist()

# Encode all abstracts in batches
embeddings = model.encode(
    abstracts,
    batch_size=64,
    show_progress_bar=True,
    normalize_embeddings=True # unit-norm: cosine sim = dot product
)
print(f"Embedding shape: {embeddings.shape}") # (N, 384)

# Save for downstream tasks (clustering, similarity search, RAG)
np.save("patent_embeddings.npy", embeddings)
df[["filing_number"]].to_csv("patent_ids.csv", index=False)
```

LDA Topic Modeling — Concept

Latent Dirichlet Allocation (LDA): discovers hidden topics in a corpus

Core idea

- Each *document* is a mixture of K topics
- Each *topic* is a distribution over vocabulary words
- Model infers both from observed word frequencies

Finance applications

- Identify themes across 10-K risk factor sections
- Track how earnings call topics evolve over time
- Construct topic-based indices (e.g., AI exposure)

Workflow

1. Clean and tokenize text (remove stop words)
2. Build document-term matrix (TF counts)
3. Fit LDA with chosen K topics
4. Inspect top words per topic
5. Assign topic weights to each document
6. Validate topics qualitatively

LDA Topic Modeling — Implementation (scikit-learn)

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation

df = pd.read_csv("earnings_calls.csv")
texts = df["clean_text"].tolist()

# Build document-term matrix
vectorizer = CountVectorizer(
    max_df=0.90, min_df=10,
    stop_words="english", max_features=5000)
dtm = vectorizer.fit_transform(texts)
vocab = vectorizer.get_feature_names_out()

# Fit LDA model
lda = LatentDirichletAllocation(
    n_components=10, random_state=42, n_jobs=-1)
lda.fit(dtm)

# Display top words for each topic
for idx, topic in enumerate(lda.components_):
    top_words = [vocab[i] for i in topic.argsort()[::-11:-1]]
    print(f"Topic {idx:2d}: {' ', ' '.join(top_words)}")

# Assign dominant topic to each document
doc_topics = lda.transform(dtm) # shape: (N_docs, 10)
df["dominant_topic"] = doc_topics.argmax(axis=1)
df["topic_weight"] = doc_topics.max(axis=1)
```


LDA Topic Modeling — Output

	Topic 3 words	Topic 3 weights	Topic 4 words	Topic 4 weights	Topic 5 words	Topic 5 weights
0	global	473.1	climat	422.0	global	783.0
1	warm	450.7	chang	401.8	warm	764.7
2	believ	101.3	legisl	123.2	gore	137.1
3	california	87.1	us	105.1	snow	123.7
4	blame	82.1	via	60.5	al	122.1

Sentiment Analysis — VADER and FinBERT

```
import pandas as pd, nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer
from transformers import pipeline

nltk.download("vader_lexicon", quiet=True)
vader = SentimentIntensityAnalyzer()

# Method 1: VADER (rule-based, fast, works for general text)
def vader_score(text):
    s = vader.polarity_scores(str(text))
    return s["compound"], int(s["compound"] > 0.05)

# Method 2: FinBERT (fine-tuned BERT, better for financial language)
finbert = pipeline("sentiment-analysis",
                    model="ProsusAI/finbert",
                    truncation=True, max_length=512)

df = pd.read_csv("earnings_sentences.csv")

# Apply VADER
df[["vader_compound", "vader_positive"]] = df["sentence"].apply(
    lambda x: pd.Series(vader_score(x)))

# Apply FinBERT (more accurate for financial text)
results = finbert(df["sentence"].tolist(), batch_size=32)
df["finbert_label"] = [r["label"] for r in results]
df["finbert_score"] = [r["score"] for r in results]
```

Sentiment Analysis — Output and Model Comparison

	reviewText	Positive	sentiment
0	one best apps acording bunch people agree bomb...	1	1
1	pretty good version game free . lot different ...	1	1
2	really cool game . bunch level find golden egg...	1	1
3	silly game frustrating , lot fun definitely re...	1	1
4	terrific game pad . hr fun . grandkids love	1	1
...	...	...	...
19995	app fricken stupid.it froze kindle wont allow ...	0	0
19996	please add !!!!! need neighbor ! ginger101...	1	1
19997	love ! game . awesome . wish free stuff house ...	1	1
19998	love love love app side fashion story fight wo...	1	1
19999	game rip . list thing make better & bull ; fir...	0	1

20000 rows x 3 columns

When to use which model:

- **VADER**: fast batch processing, social media, short texts
- **FinBERT**: financial news, earnings calls; knows that “liability” is negative in finance
- **Loughran-McDonald dictionary**: simple word-count baseline; standard in accounting research
- **LLM scoring**: highest accuracy; ask GPT-4o or Claude to rate sentiment 1–5 with reasoning

Key insight: always validate on a labeled sample before scaling up

Machine Learning with Structured Features

Machine Learning Workflow

1. Data Preparation

Load features; handle missing values; **split by time** to prevent look-ahead bias

2. Feature Engineering

Normalize continuous features (StandardScaler); encode categoricals; construct interaction terms

3. Model Training and Selection

Logistic Regression, SVM, XGBoost, Random Forest; tune hyperparameters with time-series cross-validation

4. Evaluation and Interpretation

AUC-ROC, precision/recall; compare in-sample vs. out-of-sample; SHAP for feature importance

Key warning

In finance, **always split data chronologically** — random splits leak future information

Machine Learning — Stock Return Classification (scikit-learn)

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn import metrics

df = pd.read_csv("stock_features.csv").sort_values("date") # chronological order
X = df[["open_close", "low_high", "is_quarter_end", "pe_ratio", "momentum_1m]].values
y = df["next_month_up"].values # 1 = positive return next month

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Time-ordered split: first 80% train, last 20% validate
X_train, X_val, y_train, y_val = train_test_split(
    X_scaled, y, test_size=0.20, shuffle=False)

for clf in [LogisticRegression(), SVC(probability=True), XGBClassifier()]:
    clf.fit(X_train, y_train)
    train_auc = metrics.roc_auc_score(y_train, clf.predict_proba(X_train)[:,-1])
    val_auc = metrics.roc_auc_score(y_val, clf.predict_proba(X_val)[:,-1])
    print(f"{type(clf).__name__:25s} Train AUC={train_auc:.3f} Val AUC={val_auc:.3f}")
```

Machine Learning — Results

Output:

```
LogisticRegression() :  
Training Accuracy : 0.5191709844559586  
Validation Accuracy : 0.5435330347144457
```

```
SVC(kernel='poly', probability=True) :  
Training Accuracy : 0.4718091537132988  
Validation Accuracy : 0.4451987681970885
```

```
XGBClassifier() :  
Training Accuracy : 0.7829611398963732  
Validation Accuracy : 0.5706187010078387
```

Evaluation of the model on training and the testing data.

Large Language Models and LLM APIs

What is a Large Language Model?

Architecture and training

- Based on the **Transformer** (Vaswani et al., 2017): attention mechanism over token sequences
- Pre-trained on massive text corpora via *next-token prediction*
- Fine-tuned with human feedback (RLHF) to follow instructions safely
- GPT-4: ~1.8 trillion parameters; training cost tens of millions of USD

Emergent capabilities at scale

- Multi-step reasoning and chain-of-thought
- Code generation, debugging, and explanation
- Complex instruction following in any language

What LLMs can do for finance

- Summarize long documents (10-K, earnings calls)
- Extract structured data from unstructured text
- Score sentiment and classify tone at scale
- Answer questions about domain-specific content
- Generate and interpret quantitative analysis
- Translate regulatory documents across languages

Frontier LLM Models (2025)

Model	Provider	Key Strengths	Context Window	Access
GPT-4o	OpenAI	General reasoning, vision, speed	128K tokens	API / ChatGPT
o3 / o1	OpenAI	Advanced multi-step reasoning	200K tokens	API
Claude Sonnet 4	Anthropic	Coding, analysis, long documents	200K tokens	API / Claude.ai
Claude Opus 4	Anthropic	Complex research, highest accuracy	200K tokens	API
Gemini 1.5 Pro	Google	1M context window, multimodal	1M tokens	API / AI Studio
Llama 3.1 405B	Meta	Open-source, self-hostable	128K tokens	Open weights
DeepSeek R1	DeepSeek	Strong math and coding	128K tokens	API / open weights

- Cost benchmark: GPT-4o-mini ≈\$0.15/million input tokens
- Claude Sonnet ≈\$3/million input tokens
- **Rule of thumb:** use cheaper models for batch classification; use frontier models for complex reasoning

API vs. Running Locally

API Approach (recommended for research)

- No GPU required — runs on any laptop
- Always the latest, best-performing model version
- Pay per token; free tiers for experimentation
- Enterprise security, uptime SLAs, rate limits
- Switch models with one line of code

For this course

We use **API calls** — you only need Python and an API key. All examples use the OpenAI and Anthropic SDKs.

Local / Self-hosted

- Full data privacy — nothing leaves your machine
- Works offline; one-time hardware cost
- Limited to smaller open models (e.g., Llama 3 8B)
- More setup; slower without high-end GPU
- Tools: HuggingFace Transformers, Ollama

OpenAI API — Basic Usage (pip install openai)

```
import os
from openai import OpenAI

# API key read from environment variable OPENAI_API_KEY
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system",
         "content": "You are an expert financial analyst."},
        {"role": "user",
         "content": "Identify the top 3 risk factors from this excerpt:\n\n"
                    "Apple's revenues depend significantly on iPhone sales, which are "
                    "subject to intense competition and global supply chain disruptions..."}
    ],
    temperature=0.2,      # lower = more deterministic output
    max_tokens=512
)

print(response.choices[0].message.content)
print(f"Tokens used: {response.usage.total_tokens}")
print(f"Estimated cost: ${response.usage.total_tokens * 0.0025 / 1000:.4f}")
```

Anthropic Claude API — Basic Usage (pip install anthropic)

```
import os
import anthropic

# API key read from environment variable ANTHROPIC_API_KEY
client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

message = client.messages.create(
    model="claude-sonnet-4-6",          # or "claude-opus-4-6" for harder tasks
    max_tokens=1024,
    system="You are an expert financial analyst specializing in earnings analysis.",
    messages=[
        {"role": "user",
         "content": "Analyze the sentiment and extract the top 3 themes "
                    "from this earnings call excerpt:\n\n"
                    "Revenue grew 12% year-over-year driven by strong cloud adoption. "
                    "However, macroeconomic headwinds and rising interest rates remain "
                    "key concerns for our enterprise clients..."}
    ]
)

print(message.content[0].text)
print(f"Input tokens: {message.usage.input_tokens}")
print(f"Output tokens: {message.usage.output_tokens}")
```

Prompt Engineering — Best Practices

1. Assign a clear role

- “*You are an expert CFA analyst with 20 years of experience*”
- Sets frame of reference, tone, and vocabulary

2. Be specific about the task

- *Bad*: “Tell me about this company”
- *Good*: “Identify 3 key risk factors and rate each 1–5 for severity”

3. Provide context and examples

- Include the text to analyze directly in the prompt
- Add few-shot examples for classification tasks
- Longer context window $\Rightarrow$ include more

4. Specify output format

- “*Return a JSON object with keys: sentiment, score, risks*”
- Makes downstream processing easy and reliable

5. Chain-of-thought prompting

- “*Think step by step before giving your final answer*”
- Significantly improves accuracy on complex reasoning

6. Temperature control

- 0.0–0.2: classification, extraction, structured output
- 0.6–1.0: summarization, creative writing

Structured Output — JSON Extraction at Scale (GPT-4o-mini)

```
from openai import OpenAI
import json, pandas as pd

client = OpenAI()

SYSTEM = "Extract data from earnings calls. Return ONLY valid JSON."
SCHEMA = """Extract: revenue_growth_pct (float, YoY %),
management_tone ("optimistic"/"neutral"/"cautious"),
key_concerns (list of 3 strings), guidance_raised (bool)."""

def extract_earnings_data(transcript: str) -> dict:
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        response_format={"type": "json_object"},
        messages=[
            {"role": "system", "content": SYSTEM},
            {"role": "user",
             "content": f"{SCHEMA}\n\nTranscript:\n{transcript[:2000]}"},
        ],
        temperature=0.1
    )
    return json.loads(resp.choices[0].message.content)

df = pd.read_csv("transcripts.csv")
out = pd.DataFrame([extract_earnings_data(t) for t in df["transcript"]])
pd.concat([df[["ticker", "date"]], out], axis=1).to_csv("llm_vars.csv", index=False)
```

Retrieval-Augmented Generation (RAG) — Concept

Problem: LLMs have a knowledge cutoff and cannot access private or proprietary data.

RAG Solution

Retrieve relevant document chunks at query time and include them in the prompt as context

RAG Pipeline

1. **Indexing (once):** chunk documents, encode with embedding model, store in vector DB
2. **Retrieval:** embed the user query, find top- k similar chunks
3. **Augmentation:** insert retrieved chunks into the prompt as context
4. **Generation:** LLM answers grounded in the retrieved evidence

Finance Use Cases

- Q&A over company 10-K and 10-Q filings
- Search over internal investment research
- Regulatory document analysis (IFRS, Basel)
- Earnings call Q&A with source citations
- Patent landscape analysis

RAG — Implementation (sentence-transformers + OpenAI)

```
from sentence_transformers import SentenceTransformer
import numpy as np
from openai import OpenAI

embed_model = SentenceTransformer("all-MiniLM-L6-v2")
openai_client = OpenAI()

# Step 1: Index documents (run once, store embeddings)
documents = [...] # text chunks from 10-K filings
doc_embeds = embed_model.encode(documents, normalize_embeddings=True)

def rag_answer(question: str, top_k: int = 4) -> str:
    # Step 2: Retrieve top-k most similar chunks
    q_emb = embed_model.encode([question], normalize_embeddings=True)
    sims = (doc_embeds @ q_emb.T).flatten()
    top_idx = sims.argsort()[-top_k:][::-1]
    context = "\n---\n".join(documents[i] for i in top_idx)

    # Step 3 & 4: Augment prompt and generate grounded answer
    resp = openai_client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "system",
             "content": "Answer using ONLY the provided context. Say if unsure."},
            {"role": "user",
             "content": f"Context:\n{context}\n\nQuestion: {question}"}
        ]
    )
    return resp.choices[0].message.content

print(rag_answer("What are Apple's main supply chain risks?"))
```

AI Agents

What is an AI Agent?

Definition

An **AI agent** uses an LLM as its reasoning engine to autonomously *plan and execute multi-step tasks*, selecting and using tools to interact with the world until the task is complete.

Four core components

- **LLM brain:** decides what to do next; generates reasoning traces
- **Tools:** web search, code execution, APIs, databases, file I/O
- **Memory:** short-term (context window) + long-term (vector store)
- **Action loop:** plan → act → observe → repeat

Agent vs. simple LLM call

- LLM call: one prompt → one response
- Agent: iterates until task is complete
- Agent decides *which tool* to call and *when*
- Can handle tasks requiring real-world data and computation

The ReAct Pattern — Reason and Act

ReAct (Yao et al., 2022): interleave *reasoning traces* with *tool actions*

Example Task: “What was NVIDIA’s revenue growth in fiscal 2024?”

Thought: I need to find NVIDIA’s FY2024 and FY2023 revenues to compute growth. I’ll search the web.

Action: `web_search("NVIDIA annual revenue FY2024 FY2023")`

Observation: *NVIDIA FY2024 revenue: \$60.9B; FY2023 revenue: \$26.97B*

Thought: $\text{Growth} = (60.9 - 26.97) / 26.97 = 125.8\%$. I can now answer.

Answer: NVIDIA’s revenue grew by approximately **126%** in fiscal year 2024.

- The agent iterates through multiple steps without the user directing each one
- **2025 recommendation:** use **LangGraph** for production agents (more flexible than legacy LangChain agents)

Function Calling — Giving the LLM Tools (OpenAI)

```
import json, yfinance as yf
from openai import OpenAI

client = OpenAI()

# 1. Define the tool schema
tools = [{"type": "function", "function": {
    "name": "get_stock_data",
    "description": "Get price, P/E ratio, and 52-week high for a stock ticker",
    "parameters": {"type": "object",
        "properties": {"ticker": {"type": "string"}},
        "required": ["ticker"]}}}]

# 2. Implement the tool
def get_stock_data(ticker):
    info = yf.Ticker(ticker).info
    return {"price": info.get("currentPrice"),
        "pe": info.get("trailingPE"),
        "52w_high": info.get("fiftyTwoWeekHigh")}

# 3. Let the model call the tool automatically
messages = [{"role": "user",
    "content": "Compare Apple vs Microsoft valuation metrics."}]
resp = client.chat.completions.create(
    model="gpt-4o", messages=messages, tools=tools, tool_choice="auto")

# 4. Execute tool calls and feed results back
for tc in (resp.choices[0].message.tool_calls or []):
    result = get_stock_data(**json.loads(tc.function.arguments))
    messages.append({"role": "tool", "tool_call_id": tc.id,
        "content": json.dumps(result)})
```

Building an Agent with LangGraph

```
# pip install langgraph langchain-openai langchain-community yfinance
from langgraph.prebuilt import create_react_agent
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.tools import Tool
import yfinance as yf

llm = ChatOpenAI(model="gpt-4o", temperature=0)

@tool
def get_stock_data(ticker: str) -> str:
    """Return price, P/E, and 52-week high for the given stock ticker."""
    info = yf.Ticker(ticker).info
    return (f"Price: ${info.get('currentPrice', 'N/A')}, "
            f"P/E: {info.get('trailingPE', 'N/A'):.1f}, "
            f"52W High: ${info.get('fiftyTwoWeekHigh', 'N/A')}")

# create_react_agent wraps the full ReAct loop automatically
agent = create_react_agent(llm, [DuckDuckGoSearchRun(), get_stock_data])

result = agent.invoke({"messages": [{"role": "user",
    "content": "Compare Tesla vs BYD: P/E ratios and recent news summary."}]}))
print(result["messages"][-1].content)
```

Install: `pip install langgraph langchain-openai langchain-community yfinance`

Multi-Agent Systems

Concept: multiple specialized agents collaborate on complex tasks

Popular frameworks (2025)

- **LangGraph** (LangChain): graph-based workflows with cycles and state; most flexible
- **CrewAI**: define “crews” of agents with roles and task assignments
- **AutoGen** (Microsoft): multi-agent conversations and code execution
- **Anthropic Agent SDK**: native multi-agent orchestration with Claude

Finance research example

- **Data agent:** fetches filings, prices, news
- **Analyst agent:** runs quantitative models, calculates ratios
- **Writer agent:** drafts structured investment memo
- **Critic agent:** fact-checks and flags inconsistencies

Key challenge: error propagation and reliability across agent steps

LLM and Agent Applications in Finance Research

Academic research

- LLM-based sentiment scores for 10-K risk factors — more nuanced than dictionaries
- AI-generated measures of disclosure specificity and informativeness
- Patent claim analysis for innovation measurement
- Classify manager tone as “optimistic” vs. “cautious” across thousands of calls

Alternative data analysis

- Summarize and compare analyst reports at scale
- Extract product-level mentions from retail earnings calls

Finance industry practice

- Automated due diligence report generation
- Real-time regulatory compliance checking
- Customer-facing financial Q&A (JPM, Morgan Stanley both deployed GPT-4 assistants)

Validation discipline

- Always validate LLM outputs on a labeled ground-truth sample
- Cheaper models for bulk; frontier models for difficult tasks
- Report inter-rater agreement between LLM and human coders

Smart Contracts and Blockchain Programming

Ethereum — Overview

Key milestones

- Proposed by Vitalik Buterin (2013); launched July 2015
- General-purpose, programmable blockchain (“world computer”)
- Native currency: Ether (ETH)
- **The Merge (Sept 2022):** Proof-of-Work → **Proof-of-Stake**; energy use reduced by ~99.5%
- Block time: ~12 seconds; L1 throughput: ~15–30 TPS





Figure 1: Web 1.0 vs Web 2.0 vs Web 3.0 (Source: lizard global)

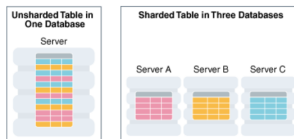
Ethereum — Architecture and Scaling

How it works

- Every node executes the same code and stores the same state
- Decentralized, censorship-resistant, trustless execution
- **Gas**: unit of computational cost; prevents infinite loops
- EIP-1559: base fee burned, validator tip; more predictable costs

Layer 2 Scaling (L2)

- Ethereum L1 is slow and expensive under load
- **Rollups** (Optimism, Arbitrum, Base): batch transactions off-chain, post proofs to L1
- Achieves thousands of TPS at a fraction of L1 cost
- Most DeFi and NFT activity has migrated to L2 networks



Smart Contracts

What is a smart contract?

- A program stored and executed on the blockchain
- Written in **Solidity** (statically typed, C++/JS-like syntax)
- Has *state variables* (stored on-chain) and *functions*
- Executes automatically when conditions are met
- Immutable once deployed (upgradeable proxy patterns are possible)

Key applications

- **DeFi**: decentralized exchanges (Uniswap), lending (Aave, Compound)
- **NFTs**: ERC-721 standard; provable digital ownership
- **DAOs**: decentralized governance via token voting
- **Stablecoins**: algorithmic minting and burning (DAI, USDC)

Solidity — A Simple Smart Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleStorage {
    uint256 private storedValue;
    address public owner;

    event ValueUpdated(address indexed updater, uint256 newValue);

    constructor() {
        owner = msg.sender;          // deployer becomes the owner
    }

    function set(uint256 x) external {
        require(msg.sender == owner, "Only owner can update value");
        storedValue = x;
        emit ValueUpdated(msg.sender, x);
    }

    function get() external view returns (uint256) {
        return storedValue;
    }
}
```

Key concepts demonstrated: state variable, constructor, require access control, events, view functions

Solidity — Key Language Concepts

Variable types

- `uint256`, `int256`: unsigned/signed 256-bit integers
- `bool`: `true` / `false`
- `address`: 20-byte Ethereum address
- `string`, `bytes`: dynamic-size data
- `mapping(K => V)`: key-value store (hash map)
- `struct`: user-defined composite type

Function visibility

- `public`: callable internally and externally
- `external`: only from outside the contract
- `internal`: current contract and derived contracts
- `private`: current contract only

State mutability

- `view`: reads state; no gas cost for pure reads
- `pure`: neither reads nor writes state
- `payable`: function can receive Ether

Error handling (atomic)

- `require(cond, "msg")`: validate inputs; reverts on failure
- `revert("msg")`: unconditional rollback
- `assert(cond)`: check internal invariants
- All state changes reverted on any failure

Solidity — Token Contract (ERC-20 Style)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleToken {
    string public name      = "PHBS Token";
    string public symbol    = "PHT";
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;

    event Transfer(address indexed from, address indexed to, uint256 amount);

    constructor(uint256 initialSupply) {
        totalSupply      = initialSupply;
        balanceOf[msg.sender] = initialSupply; // all tokens go to deployer
    }

    function transfer(address to, uint256 amount) external returns (bool) {
        require(balanceOf[msg.sender] >= amount, "Insufficient balance");
        balanceOf[msg.sender] -= amount;
        balanceOf[to]         += amount;
        emit Transfer(msg.sender, to, amount);
        return true;
    }
}
```


Modern Blockchain Development Tools

Development and Testing

- **Remix IDE** (remix.ethereum.org): browser-based Solidity editor; ideal for learning
- **Hardhat**: Node.js framework for compile, test, deploy; most widely used in 2025
- **Foundry**: Rust-based toolchain; fastest test execution; preferred for production

Note

The Mist wallet and Truffle/Ganache stack have been deprecated. Use Hardhat or Foundry for new projects.

Deployment and Interaction

- **MetaMask**: browser wallet for signing transactions (replaced deprecated Mist)
- **Ethers.js** / **web3.py**: JS/Python libraries for contract interaction
- **Etherscan**: block explorer; verify and read deployed contracts
- **Sepolia testnet**: free ETH via faucets for safe testing

Interacting with Contracts Using Python (web3.py)

```
from web3 import Web3
import json

# Connect to a local Hardhat node (or use Infura/Alchemy for mainnet)
w3 = Web3(Web3.HTTPProvider("http://127.0.0.1:8545"))
print(f"Connected: {w3.is_connected()} Latest block: {w3.eth.block_number}")

# Load compiled contract artifact (ABI + address from Hardhat deploy output)
with open("artifacts/contracts/SimpleToken.sol/SimpleToken.json") as f:
    artifact = json.load(f)

contract = w3.eth.contract(
    address="0xYourDeployedContractAddress",
    abi=artifact["abi"])

# Read state (free, no gas)
total = contract.functions.totalSupply().call()
print(f"Total supply: {total:,} PHT tokens")

# Send a transaction (requires signing with private key)
deployer = w3.eth.accounts[0]
tx_hash = contract.functions.transfer(
    "0xRecipientAddress", 1_000).transact({"from": deployer})
receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
print(f"Transfer confirmed in block {receipt['blockNumber']}")
```